

Resolución del Práctico 3

Testing de Software 2026

Tomás Rodeghiero

Índice

1. Ejercicio 1: Lectura del capítulo 6	3
2. Ejercicio 2: numberOfOccurrences con PWC	3
2.1. MDE	4
2.2. Requisitos PWC	4
2.3. Casos de prueba	4
2.4. Implementación	5
2.5. Cómo ejecutar	5
3. Ejercicio 3: intersection — análisis del MDE y BCC	6
3.1. a) y b) — C1 (idem C2)	6
3.2. c) — C3 no es completa	6
3.3. d) — C3 tampoco es disjunta	6
3.4. e) — MDE corregido	6
3.5. f) — Base Choice Coverage	7
3.6. Casos implementados	7
3.7. Implementación	8
3.8. Cómo ejecutar	8
4. Ejercicio 4: PatternIndex — PWC y detección de fallas	8
4.1. MDE	8
4.2. Requisitos y casos	9
4.3. Fallas detectadas en la implementación original	9
4.4. Corrección	10
4.5. Cómo ejecutar	11
5. Ejercicio 5: Iterator sobre ArrayList — PWC	11
5.1. Especificación de referencia	11

5.2. MDE (según consigna)	11
5.3. Requisitos y casos	11
5.4. Resultado	12
5.5. Cómo ejecutar	12

1. Ejercicio 1: Lectura del capítulo 6

El primer ejercicio es la lectura del capítulo 6 de *Introduction to Software Testing* (Ammann y Offutt, 2.^a edición), *Input Space Partitioning*.

La idea central es que en lugar de elegir casos de test “a ojo”, se modela el dominio de entradas y de ahí se derivan requisitos claros y medibles.

Los conceptos más importantes son:

- **Modelo del Dominio de Entradas (MDE).** Se identifican los parámetros y se les asocian *características* relevantes; cada característica se divide en *bloques* (clases de valores). Un MDE bien hecho cumple dos propiedades: toda entrada válida cae en algún bloque (*completitud*) y no cae en dos a la vez (*disjointness*).
- **Dos enfoques para construirlo.** *Basado en interfaz* (una característica por parámetro, mirando la sintaxis) es rápido pero ignora relaciones; *basado en funcionalidad* (características derivadas del comportamiento esperado) es más costoso pero más expresivo.
- **Criterios sobre el modelo.**
 - *Each Choice Coverage (ECC)*: cada bloque aparece al menos en un test.
 - *Pair-Wise Coverage (PWC)*: para cada par de características, todo par de bloques aparece en algún test.
 - *Base Choice Coverage (BCC)*: se elige un test base y se varía una característica a la vez.
 - *All Combinations (ACoC)*: producto cartesiano. Máxima fuerza, máximo costo.
- **PWC vs ACoC.** PWC suele dar buena detección con muchos menos casos: la mayoría de los defectos se dispara por interacción de a dos factores.
- **Restricciones e infeasibility.** No todas las combinaciones son válidas. En ACoC y PWC se descartan las combinaciones inválidas; en BCC se reemplaza un valor por uno no base hasta obtener una combinación factible.

2. Ejercicio 2: numberOfCurrences con PWC

Diseño de una batería de tests para `ListUtils.numberOfCurrences(List<Integer>l, Integer element)` aplicando *Pair-Wise Coverage* sobre un MDE basado en funcionalidad.

2.1. MDE

Cuatro características: dos verifican que los parámetros sean no nulos (precondición), las otras dos describen el comportamiento funcional sobre la lista.

- **C1 (referencia de 1):** $L0 = \text{null}$, $L1 = \text{no null}$.
- **C2 (referencia de element):** $E0 = \text{null}$, $E1 = \text{no null}$.
- **C3 (tamaño de 1, si L1):** $S0 = \text{vacía}$, $S1 = \text{no vacía}$.
- **C4 (cantidad de ocurrencias, si L1 y E1):** $O0 = 0$, $O1 = 1$, $O2 = \text{más de una}$.

Restricciones de factibilidad:

- Si $L0$, $C3$ y $C4$ no aplican (la lista nula dispara excepción antes de inspeccionar otra cosa).
- Si $E0$, $C4$ no aplica.
- Si $S0$, necesariamente $O0$ (una lista vacía no puede contener al elemento).

2.2. Requisitos PWC

Los pares factibles entre características son 20:

- R01: ($L0$, $E0$) R02: ($L0$, $E1$)
- R03: ($L1$, $E0$) R04: ($L1$, $E1$)
- R05: ($L1$, $S0$) R06: ($L1$, $S1$)
- R07: ($L1$, $O0$) R08: ($L1$, $O1$) R09: ($L1$, $O2$)
- R10: ($E0$, $S0$) R11: ($E0$, $S1$)
- R12: ($E1$, $S0$) R13: ($E1$, $S1$)
- R14: ($E1$, $O0$) R15: ($E1$, $O1$) R16: ($E1$, $O2$)
- R17: ($S0$, $O0$) R18: ($S1$, $O0$) R19: ($S1$, $O1$) R20: ($S1$, $O2$)

2.3. Casos de prueba

TC	Entrada 1	element	Salida	Cubre
TC1	null	null	IAE	R01
TC2	null	7	IAE	R02
TC3	[]	null	IAE	R03, R05, R10

TC	Entrada 1	element	Salida	Cubre
TC4	[1,2,3]	null	IAE	R03, R06, R11
TC5	[]	5	0	R04, R05, R07, R12, R14, R17
TC6	[1,2,3]	9	0	R04, R06, R07, R13, R14, R18
TC7	[1,2,3]	2	1	R04, R06, R08, R13, R15, R19
TC8	[4,1,4,4]	4	3	R04, R06, R09, R13, R16, R20

(IAE = `IllegalArgumentException`.) Hay un caso de robustez adicional (TC9) que verifica que la rutina no muta la lista: tras invocar `numberOfOccurrences` sobre `[4,1,4,4]` buscando 4, la lista debe seguir siendo `[4,1,4,4]`.

2.4. Implementación

```

1 public static int numberOfOccurrences(List<Integer> l, Integer
   element) {
2     if (l == null || element == null) {
3         throw new IllegalArgumentException("List and element must
           be non-null");
4     }
5     int count = 0;
6     for (Integer current : l) {
7         if (element.equals(current)) {
8             count++;
9         }
10    }
11    return count;
12 }

```

Primero las precondiciones (R01–R03 dependen de eso) y después un recorrido con `element.equals(current)` para que `Integer` se compare por valor y no por referencia.

2.5. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Dtest=numberOfOccurrencesTest test
```

3. Ejercicio 3: intersection — análisis del MDE y BCC

Sobre `SetUtils.intersection(Set<Integer>set1, Set<Integer>set2)`, partiendo del MDE provisto, hay que verificar las propiedades de C1, C2 y C3, corregir lo que falle y derivar una suite por *Base Choice Coverage*.

3.1. a) y b) — C1 (idem C2)

Completa y disjunta. Para `Set<Integer>set1` solo hay tres posibilidades: nulo, no nulo y vacío, o no nulo con al menos un elemento. Los bloques `b1`, `b2`, `b3` cubren esas tres situaciones y se excluyen mutuamente (un set nulo no puede ser vacío ni no vacío, y un set vacío no puede ser no vacío al mismo tiempo).

3.2. c) — C3 no es completa

Contraejemplo: `set1 = {1, 2}`, `set2 = {2, 3}`. Hay intersección no vacía (`{2}`), pero no son iguales, ninguno es subconjunto del otro y tampoco son disjuntos. Esa configuración no encaja en ningún bloque del C3 original: el solapamiento parcial queda sin cubrir.

3.3. d) — C3 tampoco es disjunta

Si “subconjunto” se interpreta de forma no estricta, el caso `set1 = set2 = {1, 2}` cae al mismo tiempo en tres bloques: representan el mismo conjunto (`b1`), `set1` es subconjunto de `set2` (`b2`) y `set2` es subconjunto de `set1` (`b3`).

3.4. e) — MDE corregido

C1 y C2 quedan igual. C3 se reescribe con la noción de subconjunto *propio* y se agrega un bloque para el solapamiento parcial:

- **C1** (validez de `set1`): `C1.b1 = null`, `C1.b2 = vacío`, `C1.b3 = no vacío`.
- **C2** (validez de `set2`): análogo a C1.
- **C3** (relación entre `set1` y `set2`), solo si ambos no nulos:
 - C3.b1: `set1 = set2`
 - C3.b2: `set1` es subconjunto *propio* de `set2`
 - C3.b3: `set2` es subconjunto *propio* de `set1`
 - C3.b4: `set1` y `set2` disjuntos

- C3.b5: solapamiento parcial (intersección no vacía y ninguno es subconjunto del otro)

Con esto C3 queda completa (todo par (`set1`, `set2`) con ambos no nulos cae en alguno de los cinco bloques) y disjunta (los cinco bloques se excluyen mutuamente).

3.5. f) — Base Choice Coverage

Caso base elegido (camino feliz): ambos sets no vacíos e iguales.

- C1 base: C1.b3 (`set1` no vacío).
- C2 base: C2.b3 (`set2` no vacío).
- C3 base: C3.b1 (`set1 = set2`).

Requisitos: el test base, más uno por cada bloque no base, ajustando cuando alguna variación deja a C3 sin aplicar.

TR	Variación respecto del base	Resultado esperado
TR-BC1	— (caso base)	{1,2,3}
TR-BC2	C1 → C1.b1 (<code>null</code>)	<code>NullPointerException</code>
TR-BC3	C1 → C1.b2 ({})	{}
TR-BC4	C2 → C2.b1 (<code>null</code>)	<code>NullPointerException</code>
TR-BC5	C2 → C2.b2 ({})	{}
TR-BC6	C3 → C3.b2 (subc. propio)	<code>set1</code>
TR-BC7	C3 → C3.b3 (superc. propio)	<code>set2</code>
TR-BC8	C3 → C3.b4 (disjuntos)	{}
TR-BC9	C3 → C3.b5 (solap. parcial)	elementos comunes

TR-BC3 y TR-BC5 son los casos donde una variación sobre C1 o C2 vuelve no aplicable a C3; según la guía para BCC con restricciones, se reemplaza la base de C3 por una combinación factible (acá, intersección vacía trivial).

3.6. Casos implementados

`IntersectionTest.java` cubre TC1–TC9, uno por TR-BC. La suite también:

- verifica que los sets de entrada no se modifican,
- verifica que el resultado es una nueva instancia (no `==` a ninguno de los sets de entrada),
- agrega un borde extra con ambos sets vacíos.

3.7. Implementación

```
1 public static Set<Integer> intersection(Set<Integer> set1, Set<
   Integer> set2) {
2     Objects.requireNonNull(set1, "set1 must be non-null");
3     Objects.requireNonNull(set2, "set2 must be non-null");
4
5     Set<Integer> result = new HashSet<>(set1);
6     result.retainAll(set2);
7     return result;
8 }
```

`Objects.requireNonNull` respeta la especificación de `NullPointerException`, y la copia previa a `retainAll` garantiza tanto la no mutación de `set1` como la nueva instancia de retorno.

3.8. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Dtest=IntersectionTest test
```

4. Ejercicio 4: PatternIndex — PWC y detección de fallas

Modelo y suite para `PatternIndex.patternIndex(String subject, String pattern)` con *Pair-Wise Coverage*, más las correcciones que aplicaron a la implementación original.

4.1. MDE

- **C1** (referencia de `subject`): C1.b1 = null, C1.b2 = no null.
- **C2** (referencia de `pattern`): C2.b1 = null, C2.b2 = no null.
- **C3** (tamaño de `subject`, si C1.b2): C3.b1 = vacío, C3.b2 = no vacío.
- **C4** (tamaño de `pattern`, si C2.b2): C4.b1 = vacío, C4.b2 = 1 carácter, C4.b3 = 2 o más.
- **C5** (posición del resultado, si C1.b2 y C2.b2): C5.b1 = índice 0, C5.b2 = intermedio, C5.b3 = final, C5.b4 = -1.

Restricciones:

- Si `subject` o `pattern` es null: precondition violada (`IllegalArgumentException`); C3–C5 no aplican.

- Si `pattern` es vacío (C4.b1): por especificación, devuelve 0 (cubre C5.b1).
- Si `subject` es vacío (C3.b1) y `pattern` no lo es (C4.b2 o C4.b3): el resultado es -1 (cubre C5.b4).

4.2. Requisitos y casos

La suite implementa 15 casos sobre los 20 requisitos R01–R20:

TC	subject	pattern	Salida	Cubre
TC1	null	null	IAE	R01
TC2	null	.a"	IAE	R02
TC3	.abc"	null	IAE	R03
TC4			0	R04, R05, R11
TC5	.abc"		0	R04, R06, R12
TC6		.a"	-1	R04, R07, R15
TC7	.abc"	.a"	0	R04, R08, R13
TC8	.abc"	"b"	1	R04, R08, R14
TC9	.abc"	ç"	2	R04, R08, R16
TC10	.abcde"	.ab"	0	R04, R09, R17
TC11	.abcde"	çd"	2	R04, R09, R18
TC12	.abcde"	"de"	3	R04, R09, R19
TC13	.abcde"	"fg"	-1	R04, R09, R20
TC14	.abc"	"z"	-1	R04, R08, R15
TC15		.ab"	-1	R04, R10, R20

4.3. Fallas detectadas en la implementación original

Los tests descubrieron dos defectos:

1. **No validaba null.** La rutina hacía `subject.length()` y `pattern.length()` sin chequear, así que TC1–TC3 disparaban `NullPointerException` en lugar del `IllegalArgumentException` que documenta el Javadoc.
2. **Pattern vacío.** Con `pattern.length() == 0`, la guarda `iSub + patternLen - 1 < subjectLen` queda como `iSub - 1 < subjectLen`, verdadera al menos para `iSub = 0`. Dentro del cuerpo se ejecuta `pattern.charAt(0)`, que dispara `StringIndexOutOfBoundsException` en vez de devolver 0.

4.4. Corrección

Agregué las dos guardas al principio del método:

```
1 public static int patternIndex(String subject, String pattern) {
2     if (subject == null || pattern == null) {
3         throw new IllegalArgumentException("subject and pattern
4             must be non-null");
5     }
6
7     final int NOTFOUND = -1;
8     int iSub = 0, rtnIndex = NOTFOUND;
9     boolean isPat = false;
10    int subjectLen = subject.length();
11    int patternLen = pattern.length();
12
13    if (patternLen == 0) {
14        return 0;
15    }
16
17    while (isPat == false && iSub + patternLen - 1 < subjectLen)
18    {
19        if (subject.charAt(iSub) == pattern.charAt(0)) {
20            rtnIndex = iSub;
21            isPat = true;
22            for (int iPat = 1; iPat < patternLen; iPat++) {
23                if (subject.charAt(iSub + iPat) != pattern.charAt
24                    (iPat)) {
25                    rtnIndex = NOTFOUND;
26                    isPat = false;
27                    break;
28                }
29            }
30            iSub++;
31        }
32    }
33    return rtnIndex;
34 }
```

El resto del algoritmo de búsqueda es correcto, así que con las dos guardas alcanza para que los 15 casos pasen.

4.5. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Dtest=PatternIndexTest test
```

5. Ejercicio 5: Iterator sobre ArrayList — PWC

Aplicación de ISP sobre la interfaz `java.util.Iterator`, usando `ArrayList` como implementación de referencia (`Iterator` es interfaz, no se instancia directamente).

5.1. Especificación de referencia

De Oracle Java SE 7 (<https://docs.oracle.com/javase/7/docs/api/java/util/Iterator.html>):

- `hasNext()`: devuelve `true` si quedan elementos.
- `next()`: devuelve el siguiente; lanza `NoSuchElementException` si no hay más.
- `remove()`: operación opcional. Si no está soportada, lanza `UnsupportedOperationException`. Si lo está, lanza `IllegalStateException` si no hubo `next()` previo, o si ya se llamó `remove()` después del último `next()`.

5.2. MDE (según consigna)

- C1: “hay más elementos” $\in \{\text{true}, \text{false}\}$.
- C2: “`next()` devuelve no nulo” $\in \{\text{true}, \text{false}\}$.
- C3: “`remove()` está soportado” $\in \{\text{true}, \text{false}\}$.
- C4: “precondición de `remove()` satisfecha” $\in \{\text{true}, \text{false}\}$.

Decisiones para instanciar:

- Para C3 = `true`: `new ArrayList<>(...).iterator()`, que soporta `remove()`.
- Para C3 = `false`: `Collections.unmodifiableList(...).iterator()`; la colección subyacente sigue siendo un `ArrayList`, pero el iterador expuesto no soporta `remove()`.

5.3. Requisitos y casos

La suite cubre 13 requisitos:

TC	R	Escenario
TC1	R01	<code>hasNext()</code> = <code>true</code> sobre iterador con datos.

TC	R	Escenario
TC2	R02	<code>hasNext()</code> = false sobre iterador vacío.
TC3	R03	<code>hasNext()</code> = false sobre iterador agotado.
TC4	R04	<code>next()</code> devuelve un valor no nulo.
TC5	R05	<code>next()</code> devuelve <code>null</code> (la lista contiene un <code>null</code>).
TC6	R06	<code>next()</code> sobre iterador vacío → <code>NoSuchElementException</code> .
TC7	R07	<code>next()</code> sobre iterador agotado → <code>NoSuchElementException</code> .
TC8	R08	<code>remove()</code> no soportado (<code>unmodifiableList</code>) → UOE.
TC9	R09	<code>remove()</code> soportado pero sin <code>next()</code> previo → ISE.
TC10	R10	<code>remove()</code> sobre iterador vacío sin <code>next()</code> previo → ISE.
TC11	R11	<code>remove()</code> elimina el último retornado, con elementos restantes.
TC12	R12	<code>remove()</code> elimina el último retornado, sin elementos restantes.
TC13	R13	Doble <code>remove()</code> consecutivo → ISE.

5.4. Resultado

No encontré fallas. El iterador de `ArrayList` (y la vista de `unmodifiableList` para el caso `C3 = false`) se comporta según la especificación en todos los escenarios modelados.

5.5. Cómo ejecutar

```
mvn -Dmaven.repo.local=.m2 -Dtest=IteratorArrayListTest test
```